

SYSTEMS DESIGN

Software Engineering II

Author: Eng. Liliana Marcela Olarte, M.Sc.
lmolartem@unal.edu.co

Author: Eng. Carlos Andrés Sierra, M.Sc.
casierrav@unal.edu.co

Computer Engineering Program
School of Engineering
Universidad Nacional de Colombia

2026-I

Outline

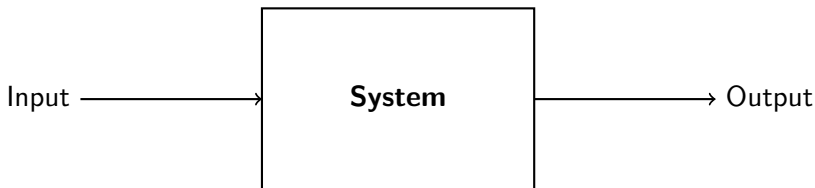
- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design
- 5 Reference Architectures

Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design
- 5 Reference Architectures

What is a System?

- A **system** is a **set** of **interacting components** that *work* together to **achieve** a common goal.
- A **system** is a **collection** of **elements** that are *organized* in a specific way.
- A **system** is a **structure** that is **designed** to *perform* a specific function.



Systems Analysis Process

- **Systems analysis** is the *process* of **studying** a **system** in order to identify its **components**, **interactions**, and **goals**.
- **Systems analysis** is the *process* of **understanding** how a **system works** and how it can be **improved**.
- **Systems analysis** is the *first step* in the **systems development lifecycle**.

5 Essential Phases of SDLC



Planning & Analysis



Designing the System Architecture



Development & Coding



Testing & Quality Assurance



Deployment & Maintenance

Systems Analysis Techniques

- **Systems analysis** uses a variety of techniques to study a system.
- It includes interviews, surveys, observations, and document analysis.
- It also includes data modeling, process modeling, and requirements analysis.

Systems Analysis Techniques

- **Systems analysis** uses a variety of techniques to study a system.
- It includes interviews, surveys, observations, and document analysis.
- It also includes data modeling, process modeling, and requirements analysis.

Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design
- 5 Reference Architectures

Basics of Object-Oriented Design I

- **Object-oriented** has become one of the **most traditional** and popular **paradigms** in **software development**.
- It is based on the concept of **objects**, which can contain data, in the form of **fields** (often known as *attributes* or *properties*), and code, in the form of **procedures** (often known as *methods*).

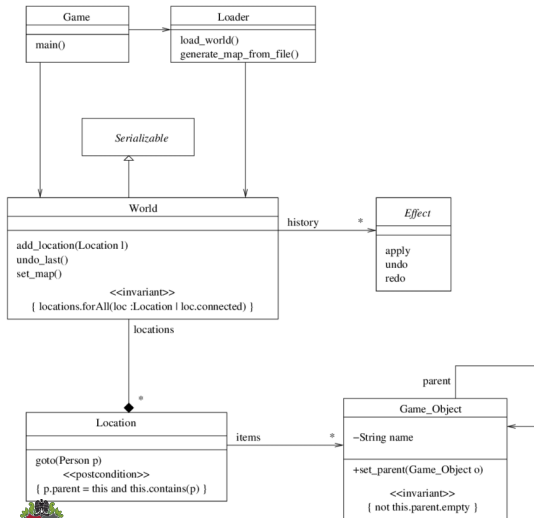
ClassName
+ attribute1 : Type1 + attribute2 : Type2 + attribute3 : Type3
+ method1() + method2() + method3()

Basics of Object-Oriented Design I

- **Object-oriented** has become one of the **most traditional** and popular **paradigms** in **software development**.
- It is based on the concept of **objects**, which can contain data, in the form of **fields** (often known as *attributes* or *properties*), and code, in the form of **procedures** (often known as *methods*).

ClassName
+ attribute1 : Type1 + attribute2 : Type2 + attribute3 : Type3
+ method1() + method2() + method3()

Basics of Object-Oriented Design II



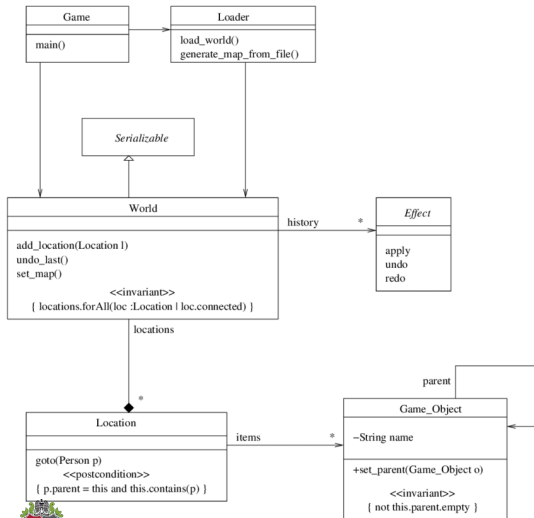
- The idea is to design a **system modularly**, and to make it **easier** to **maintain**, and to understand. Also the idea is **emphasize** the **reuse of code**.

- The **main principles of OOD** are:

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism



Basics of Object-Oriented Design II

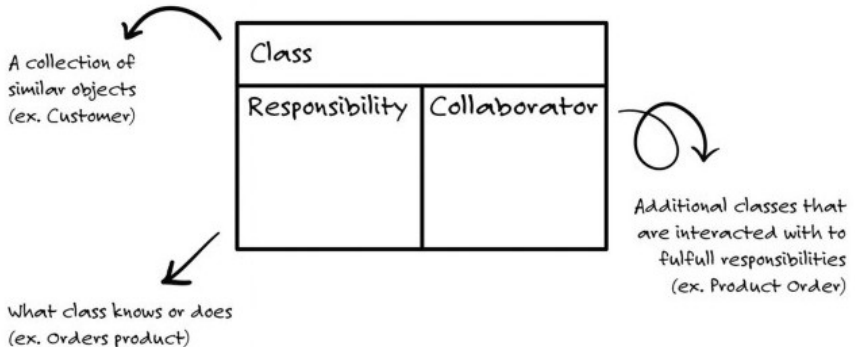


- The idea is to design a **system modularly**, and to make it **easier** to **maintain**, and to understand. Also the idea is **emphasize** the **reuse of code**.
- The **main principles** of **OOD** are:
 - Encapsulation
 - Abstraction
 - Inheritance
 - Polymorphism



Class-Responsability-Collaboration Cards (CRC)

The **CRC cards** are a **brainstorming tool** used in the **design** of **object-oriented software**.



Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles**
- 4 Domain-Driven Design
- 5 Reference Architectures

SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only **one reason to change**.
- **Open/Closed Principle (OCP):** A class should be **open** for extension, but **closed** for modification.
- **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their **subtypes** without altering the correctness of that program.
- **Interface Segregation Principle (ISP):** A client **should never be forced** to implement an interface that it doesn't use or clients **shouldn't be forced** to depend on methods they do not use.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules. Both should **depend on** abstractions. Abstractions should not depend on details. Details **should depend on** abstractions.

SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only **one reason to change**.
- **Open/Closed Principle (OCP):** A class should be **open** for extension, but **closed** for modification.
- **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their **subtypes** without altering the correctness of that program.
- **Interface Segregation Principle (ISP):** A client **should never be forced** to implement an interface that it doesn't use or clients **shouldn't be forced** to depend on methods they do not use.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules. Both should **depend on** **abstractions**. Abstractions should not depend on details. Details **should depend on** **abstractions**.



SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only **one reason to change**.
- **Open/Closed Principle (OCP):** A class should be **open** for extension, but **closed** for modification.
- **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their **subtypes** without altering the correctness of that program.
- **Interface Segregation Principle (ISP):** A client **should never be forced** to implement an interface that it doesn't use or clients **shouldn't be forced** to depend on methods they do not use.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules. Both should **depend on abstractions**. Abstractions should not depend on details. **Details should depend on abstractions**.



SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only **one reason to change**.
- **Open/Closed Principle (OCP):** A class should be **open** for extension, but **closed** for modification.
- **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their **subtypes** without altering the correctness of that program.
- **Interface Segregation Principle (ISP):** A client **should never be forced** to implement an interface that it doesn't use or clients **shouldn't be forced** to depend on methods they do not use.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules. Both should **depend on abstractions**. Abstractions should not depend on details. **Details should depend on abstractions**.



SOLID Principles

- **Single Responsibility Principle (SRP):** A class should have only **one reason to change**.
- **Open/Closed Principle (OCP):** A class should be **open** for extension, but **closed** for modification.
- **Liskov Substitution Principle (LSP):** Objects in a program should be replaceable with instances of their **subtypes** without altering the correctness of that program.
- **Interface Segregation Principle (ISP):** A client **should never be forced** to implement an interface that it doesn't use or clients **shouldn't be forced** to depend on methods they do not use.
- **Dependency Inversion Principle (DIP):** High-level modules should not depend on low-level modules. Both should **depend on abstractions**. Abstractions should not depend on details. **Details should depend on abstractions**.



Good Practices

- **Composition over Inheritance:** Inheritance should be used **only when there is a clear relationship** between the base class and the derived class. In other cases, **composition** should be used. Inheritance is a powerful tool, but it is not always the best tool for the job. Inheritance is a way to achieve polymorphism, but it is **not the only way to achieve polymorphism**.
- **Code to Interfaces, not Implementations:** This principle is about designing your classes so that they **depend on interfaces** rather than concrete classes.

Demo time!

Good Practices

- **Composition over Inheritance:** Inheritance should be used **only when there is a clear relationship** between the base class and the derived class. In other cases, **composition** should be used. Inheritance is a powerful tool, but it is not always the best tool for the job. Inheritance is a way to achieve polymorphism, but it is **not the only way to achieve polymorphism**.
- **Code to Interfaces, not Implementations:** This principle is about designing your classes so that they **depend on interfaces** rather than concrete classes.

Demo time!

Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design**
- 5 Reference Architectures

Basics of Domain-Driven Design I

- **DDD** is focusing on the **core domain** and **domain logic**, it is a way of **thinking** aimed at accelerating software projects that have to **deal with complicated domains**.
- The essential **terms** of **DDD** are *context, model, ubiquitous language, bounded context, and business logic in layers*.
- **DDD** is a set of **principles** and **patterns** that help to **design** a **system** ensuring alignment with the **real-world business needs**.



Figure: Prompt: Draw a soccer coach teaching robots soccer players.

Basics of Domain-Driven Design II

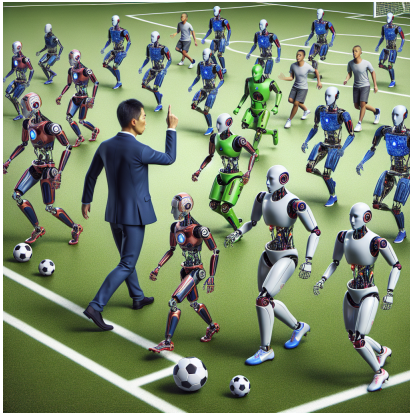


Figure: Prompt: Draw a soccer coach teaching robots soccer players.



- The **main principles** of **DDD** are:
 - **Focus** on the **core domain**.
 - Base **complex designs** on **models** of the **domain**.
 - Constantly **collaborate** with **domain experts**.
 - Develop a **knowledge-rich model**.
- The **business logic** in **layers** is shown as follows:
 - **Domain Layer**.
 - **Application Layer**.
 - **Presentation Layer**.
 - **Infrastructure Layer**.

Basics of Domain-Driven Design II

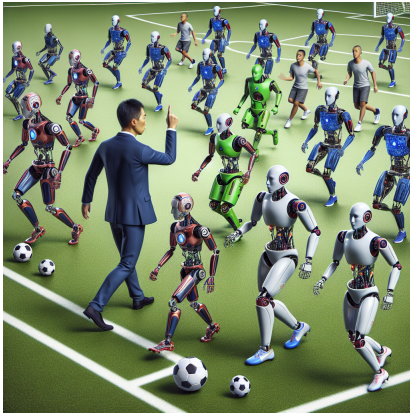
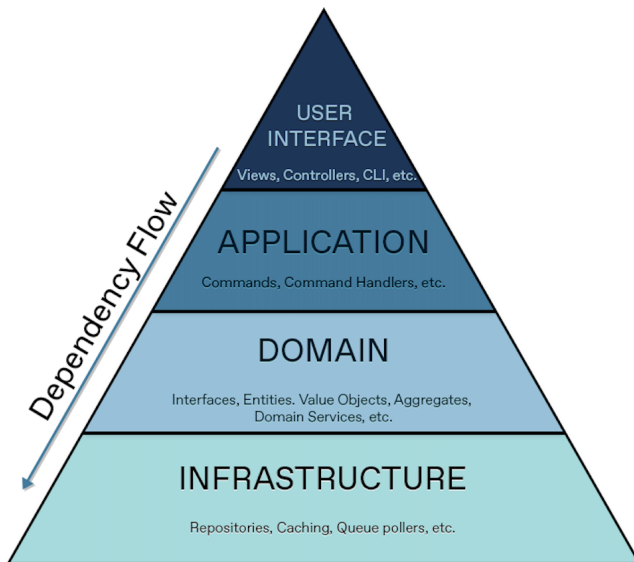


Figure: Prompt: Draw a soccer coach teaching robots soccer players.



- The **main principles** of **DDD** are:
 - Focus on the **core domain**.
 - Base **complex designs** on **models** of the **domain**.
 - Constantly **collaborate** with **domain experts**.
 - Develop a **knowledge-rich model**.
- The **business logic** in **layers** is showed as follows:
 - **Domain** Layer.
 - **Application** Layer.
 - **Presentation** Layer.
 - **Infrastructure** Layer.

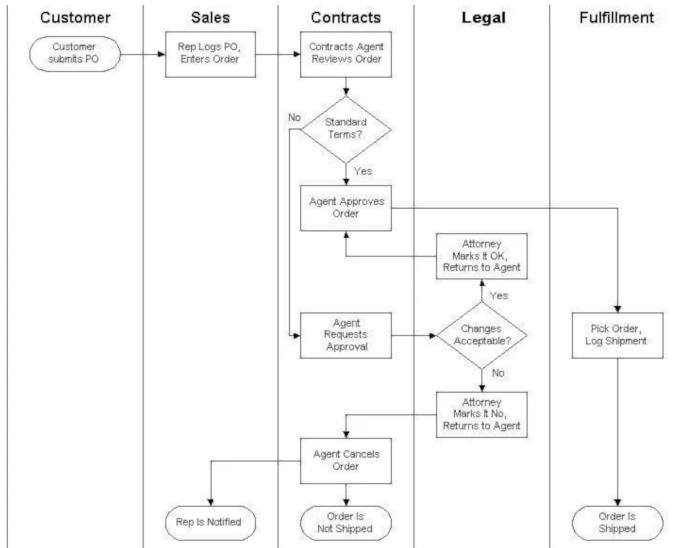
Business Logic in Layers



Process Models

- A **Process Model** is a representation of a **process** that shows the sequence of steps and the **relationships** between them.
- **Process models** are used to **analyze**, **design**, and **improve** processes.
- Examples of **process models** include flowcharts, data flow diagrams, activity diagrams, business process model and notation (BPMN), petri nets, state diagrams, among others.

Business Process Model and Notation (BPMN)



Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design
- 5 Reference Architectures

Design Before Code

- **Design** should come **before coding**.
- Jumping into *code without a plan* leads to **confusion** and **rework**.
- **Good design** **clarifies** the problem and **guides** the solution.

- Requirements must be well **understood before design**.
- Ask questions, clarify ambiguities, and document all requirements.
- Requirements define the **scope** and **direction** of the design.

- **Design** should be **driven by the problem**, not by technology.
- Focus on what **needs** to be solved, *not just* how to implement it.
- Use the **problem statement** to identify **key objects** and their **relationships**.



Design Before Code

- **Design** should come **before coding**.
 - Jumping into *code without a plan* leads to **confusion** and **rework**.
 - **Good design** **clarifies** the problem and **guides** the solution.
-
- **Requirements** must be well **understood before design**.
 - Ask questions, clarify ambiguities, and document all requirements.
 - **Requirements** define the **scope** and **direction** of the design.
-
- **Design** should be **driven by the problem**, not by technology.
 - Focus on what **needs** to be solved, *not just* how to implement it.
 - Use the **problem statement** to identify **key objects** and their *relationships*.

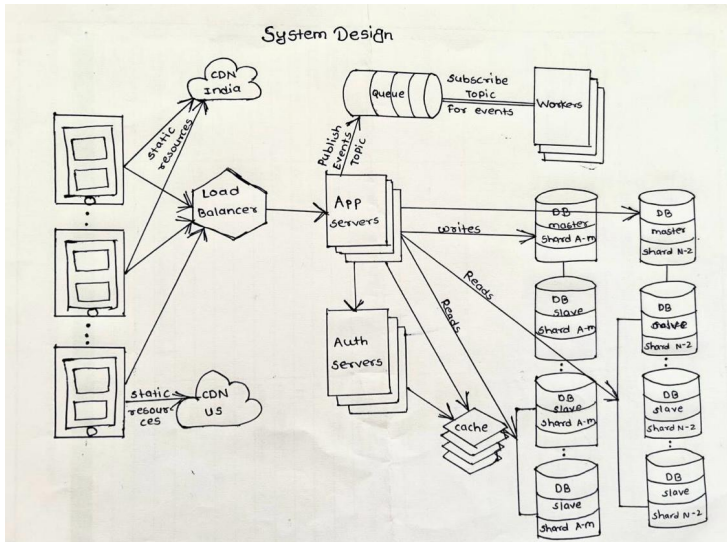


Design Before Code

- **Design** should come **before coding**.
 - Jumping into *code without a plan* leads to **confusion** and **rework**.
 - **Good design** **clarifies** the problem and **guides** the solution.
-
- **Requirements** must be well **understood before design**.
 - Ask questions, clarify ambiguities, and document all requirements.
 - **Requirements** define the **scope** and **direction** of the design.
-
- **Design** should be **driven by the problem**, not by technology.
 - Focus on what **needs** to be solved, *not just* how to implement it.
 - Use the **problem statement** to identify **key objects** and their *relationships*.



Systems Design applied to Software Architectures



Conceptual Design and Technical Design

- **Conceptual Design:** What the **system should do**, using high-level models. For example: Mockups.
- **Technical Design:** How the **system will be implemented**, using detailed diagrams and specifications.
- Both are **essential** for a **successful** software project.

Conceptual Design and Technical Design

- **Conceptual Design:** What the **system should do**, using high-level models. For example: Mockups.
- **Technical Design:** How the **system will be implemented**, using detailed diagrams and specifications.
- Both are **essential** for a **successful** software project.

Conceptual Design and Technical Design

- **Conceptual Design:** What the **system should do**, using high-level models. For example: Mockups.
- **Technical Design:** How the **system will be implemented**, using detailed diagrams and specifications.
- Both are **essential** for a **successful** software project.

What is a System Architecture?

- A **system architecture** is the *structure* of a system that *defines* its **components**, **interactions**, and **relationships**.
- A **system architecture** is the *blueprint* of a system that *guides* its **development** and **implementation**.
- A **system architecture** is the foundation of a **system** that **ensures** that it **meets** the **needs** of its **users**.
- There are *several types* of **system architectures** that are *used* in systems development.
- They include **monolithic**, **client-server**, **peer-to-peer**, and **distributed architectures**.
- Each *type* of **architecture** has its own **advantages** and **disadvantages** that *depend* on the specific requirements of the **system**.

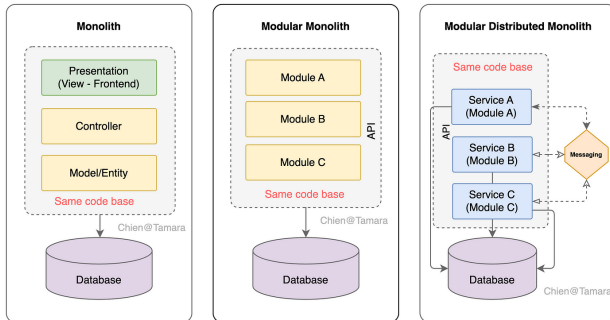
What is a System Architecture?

- A **system architecture** is the *structure* of a system that *defines* its **components**, **interactions**, and **relationships**.
- A **system architecture** is the *blueprint* of a system that *guides* its **development** and **implementation**.
- A **system architecture** is the foundation of a **system** that **ensures** that it **meets** the **needs** of its **users**.
- There are *several types* of **system architectures** that are *used* in systems development.
- They include **monolithic**, **client-server**, **peer-to-peer**, and **distributed architectures**.
- Each *type* of **architecture** has its own **advantages** and **disadvantages** that *depend* on the specific requirements of the **system**.



Monolithic System Architecture

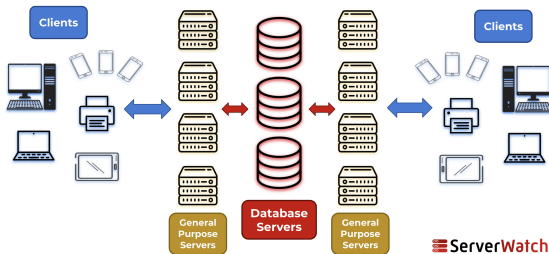
- A **monolithic system architecture** is a **single-tier architecture** that consists of a **single unit** that performs all the functions of the **system**.
- It is **simple**, **easy** to **develop**, and **maintain**, but it is **not scalable** and **flexible**. It is typically used for small systems that do not require *high performance or reliability*.



Client-Server System Architecture

- A **client-server system architecture** is a **two-tier architecture** that consists of a **client** and a **server** that communicate with each other over a **network**.
- It is **scalable**, **flexible**, and **efficient**, but it is **complex** and **difficult** to **develop** and **maintain**. It is used for *medium to large systems* that require high performance and reliability.

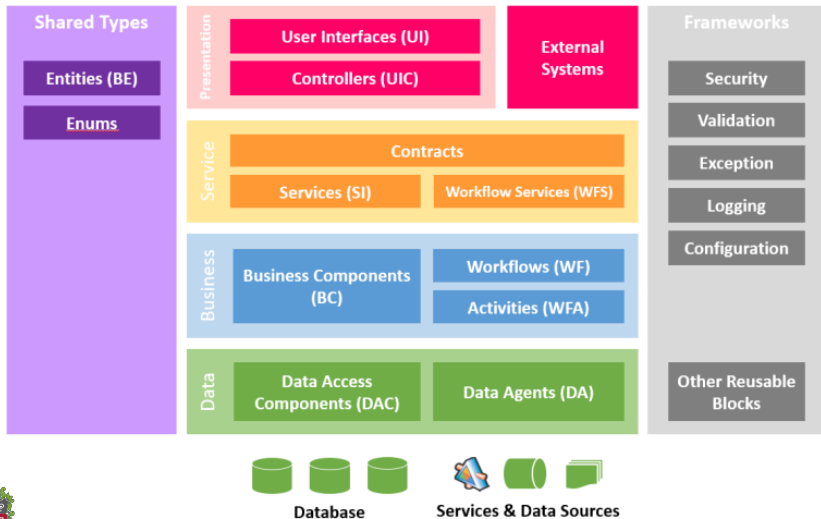
The Client-Server Model



Software Architectures

- A **software architecture** is a **high-level structure** of a **software system** that defines its **components**, **interactions**, and **relationships**.
- A **software architecture** is the *blueprint* of a **software system** that guides its **development** and **implementation**.
- A **software architecture** is the foundation of a **software system** that ensures it meets the needs of its users.

Layered Architecture Pattern



Microservices Architecture Pattern



Microservices Architecture on Google Cloud

#GCPsketchnotes



@PVERGADIA



THECLOUDGIRL.DEV

09.12.2021

What are the benefits of microservices?



PROS



Greater Flexibility



Fast time to market & fast dev cycles



Small dev teams & code base



Fault isolation



Cloud ready



Mix of technologies



Better scalability



Easy to create CI/CD pipelines



Platform & language agnostic

It's been hard to update & maintain our monolithic website.

We should migrate to...

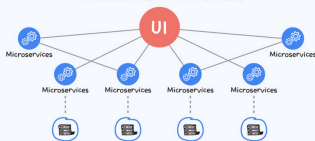
Microservices!

What's the difference between monolith & microservices architecture?

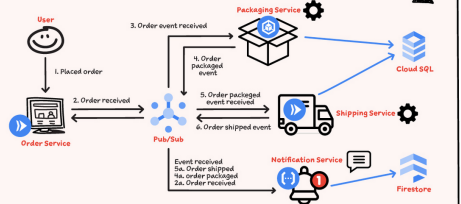
MONOLITHIC ARCHITECTURE



MICROSERVICES ARCHITECTURE



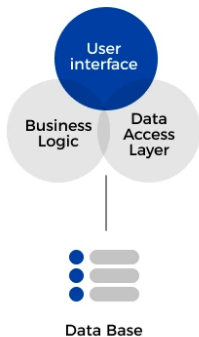
EXAMPLE OF MICROSERVICES ARCHITECTURE IN GOOGLE CLOUD



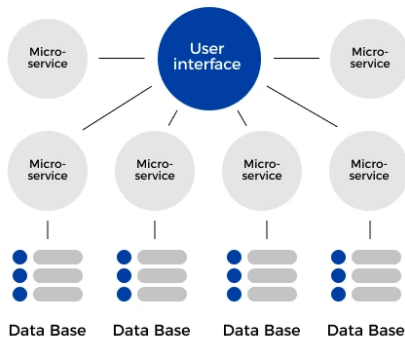
UNIVERSIDAD
NACIONAL
DE COLOMBIA

Monolithic vs. Microservices Architectures

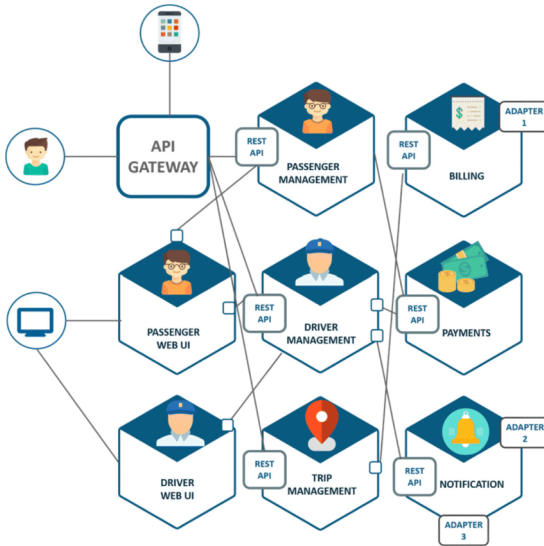
MONOLITHIC ARCHITECTURE



MICROSERVICE ARCHITECTURE



Case Study: Uber Microservices



Source: Kappagantula 2018

Outline

- 1 Systems Thinking
- 2 Object-Oriented Design
- 3 SOLID Principles
- 4 Domain-Driven Design
- 5 Reference Architectures

Thanks!

Questions?